

KAAYAA

AI & Agent Observability

“**KAAYAA**” name was chosen to represent the idea of a silent guardian that constantly watches over a system. In my vision, KAAYAA is imagined as a vigilant presence like a person who quietly observes every action, identifies unusual behavior, and protects against potential problems before they become serious. This concept aligns perfectly with the project's purpose of AI-powered observability and monitoring.

The inspiration comes from the idea of the **human subconscious**. Just as the subconscious mind continuously monitors our surroundings, detects danger, and triggers protective responses without requiring conscious effort, KAAYAA continuously observes applications, services, and AI workflows in the background. It analyzes system behavior, detects anomalies, identifies failures, and provides insights to help maintain reliability and performance.

Designed By

Abhishek Ugare

Contents

1. Introduction	04
1.1 Introduction	
1.1.1 Sub Points	
1.2 Need of Project	
2. Literature Surveys	08
2.1 Literature Survey	
2.2 Problem Statement	
2.3 Problem Solution	
3. Working Models.....	10
3.1 Related Work	
3.2 System Requirements	
3.2.1 Software Requirements	
3.2.2 Hardware Requirements	
3.3 System Design	
3.3.1 Technical Diagrams	
4. Technical Content	15
4.1 Details of front end of the project	
4.2 Details of back end of the project	
4.3 Details of Agentic AI of the project	
4.4 Connection between front end and back end, Entire details	
5. Implementation	19
5.1 Implementation Screenshot/ Snaps	
5.2 Output	
5.3 System Testing and Test results table	
5.3.1 Results and Discussion	

6. Conclusion	29
6.1 Applications or Advantages of project	
6.2 Limitations or Disadvantages of project	
6.3 Future Work / Future scope	
6.4 Conclusion	

Chapter 1. Introduction

1.1 Introduction

Artificial Intelligence (AI) has rapidly evolved from simple rule-based systems to intelligent autonomous agents capable of performing complex tasks such as reasoning, planning, decision-making, and interacting with external services. Modern AI applications often rely on multiple interconnected components, including Large Language Models (LLMs), APIs, databases, workflow automation tools, and cloud services. These interconnected systems, known as **Agentic AI systems**, provide powerful capabilities but also increase the complexity of monitoring and debugging application behavior.

Unlike traditional software applications, AI workflows are dynamic in nature. A single user request may pass through several stages, such as prompt processing, workflow orchestration, API calls, AI model inference, database operations, and response generation. If an error occurs or the response time increases, identifying the exact cause becomes challenging using conventional logging techniques. Traditional logs provide isolated events but fail to present the complete execution path of a request across multiple services.

To overcome this challenge, **observability** has become an essential aspect of modern AI system development. Observability enables developers to understand the internal behavior of applications by collecting and analyzing **traces, metrics, and logs**. It helps monitor system performance, detect failures, identify bottlenecks, measure AI model latency, and improve overall reliability. Distributed tracing, in particular, allows developers to follow a request from its origin through every service involved in processing it, making debugging and performance optimization significantly easier.

KAAYAA is an AI-powered observability platform developed to provide end-to-end monitoring and analysis of AI workflows. The project integrates **OpenTelemetry** for instrumentation, **n8n** for workflow automation, and **SigNoz** as the observability platform to capture and visualize telemetry data in real time. Every stage of an AI workflow from receiving a user request to generating the final response is instrumented using OpenTelemetry, producing distributed traces and performance metrics that are exported to SigNoz for storage, analysis, and visualization.

In addition to leveraging SigNoz, KAAYAA includes a **custom-built analytics dashboard** designed to present observability data in a simple and interactive manner. The dashboard provides insights into workflow execution, AI response times, request success and failure rates, service latency, error occurrences, token consumption, and overall system performance. These visualizations enable developers and system administrators to quickly identify issues, monitor application health, and make informed decisions for optimization.

1.1.1 Sub Points

The KAAYAA project consists of several integrated components that work together to provide complete observability for AI workflows. Each component has a specific responsibility in monitoring, collecting, processing, and visualizing telemetry data.

- **Agentic AI Workflow**

The core of the project is an Agentic AI workflow that automates the execution of user requests. Instead of performing a single AI task, the workflow coordinates multiple steps such as receiving user input, processing prompts, invoking AI models, interacting with APIs or databases, and generating the final response. This workflow is designed using **n8n**, which provides a visual and scalable workflow automation environment.

- **OpenTelemetry Instrumentation**

OpenTelemetry is an open-source observability framework used to instrument the application. It automatically collects telemetry data during workflow execution. Every important operation is recorded as a **span**, and together these spans form a **distributed trace** representing the complete lifecycle of a request.

- **SigNoz Observability Platform**

SigNoz serves as the centralized observability platform where telemetry data is stored, processed, and visualized. It receives traces and metrics exported by OpenTelemetry using the OTLP protocol.

- **n8n Workflow Automation**

n8n is used to automate the AI workflow and coordinate communication between different services. It acts as the orchestration engine that connects AI models, APIs, databases, and telemetry components into a single workflow.

- **Custom Analytics Dashboard**

KAAYAA includes a custom-built dashboard that presents observability data in an easy-to-understand graphical format. Instead of relying only on the default SigNoz interface, the dashboard provides simplified analytics for developers and administrators.

- **End-to-End Distributed Tracing**

One of the key features of KAAYAA is end-to-end distributed tracing. Every request is tracked from the moment it enters the system until the final response is delivered. This allows developers to visualize the complete execution path and identify delays or failures at any stage.

- **Performance Monitoring**

The platform continuously monitors system performance using metrics collected through OpenTelemetry. These metrics provide valuable insights into the health and efficiency of the AI application.

- **Error Detection and Debugging**

KAAYAA records exceptions and failed workflow executions in real time. When an error occurs, the associated trace contains detailed information about the failure, making debugging much faster than traditional log-based approaches.

1.2 Need of Project

Artificial Intelligence (AI) applications are becoming increasingly complex, with modern systems relying on multiple interconnected services such as Large Language Models (LLMs), workflow automation platforms, databases, APIs, and cloud infrastructure. A single user request often passes through several processing stages before generating a response. As the complexity of these systems increases, monitoring their behavior and identifying issues becomes a significant challenge.

Traditional monitoring methods, which primarily rely on logs and basic performance metrics, are no longer sufficient for AI-driven applications. Logs provide information about individual events but do not show the complete flow of a request across multiple services. When an AI workflow experiences delays, failures, or unexpected behavior, developers often spend considerable time manually tracing the execution process to identify the root cause. This increases debugging time, reduces system reliability, and affects the overall user experience.

One of the major challenges in AI applications is the lack of **end-to-end visibility**. Developers need to know how a request moves through different components, how long each stage takes, which AI model was used, how many tokens were consumed, and where failures occurred. Without this information, optimizing AI performance and maintaining production-ready systems becomes difficult.

Observability addresses these challenges by providing comprehensive insights into the internal behavior of an application through **distributed traces, metrics, and logs**. By collecting telemetry data from every stage of the workflow, developers can monitor application

performance in real time, detect anomalies, identify bottlenecks, and resolve issues more efficiently.

The **KAAYAA** project was developed to fulfill this need by providing a complete observability solution for AI workflows. It combines **OpenTelemetry** for instrumentation, **n8n** for workflow orchestration, **SigNoz** for telemetry collection and visualization, and a **custom analytics dashboard** for simplified monitoring. This integrated approach enables developers to track every AI request from its initiation to completion while continuously monitoring system health and performance.

Chapter 2. Literature Survey

2.1 Literature Survey

The rapid growth of Artificial Intelligence (AI) and Large Language Models (LLMs) has transformed the way software applications are developed and deployed. Modern AI systems are no longer limited to generating responses; they are capable of reasoning, planning, interacting with external tools, and executing complex workflows. These systems, often referred to as **Agentic AI**, involve multiple interconnected components such as workflow automation platforms, AI models, databases, APIs, and cloud services. While these capabilities have significantly improved application intelligence, they have also introduced new challenges in monitoring, debugging, and maintaining AI applications.

Traditional monitoring techniques primarily rely on application logs, server metrics, and error reports. Although these methods provide useful information about system health, they do not offer complete visibility into how a request travels across multiple services. In distributed AI systems, a single user request may pass through several components before producing a response. If any stage experiences delays or failures, identifying the root cause using conventional logging becomes difficult and time-consuming.

To address these limitations, the concept of **observability** has gained significant importance in modern software engineering. Observability is the ability to understand the internal state of a system by analyzing its external outputs, including **traces, metrics, and logs**. Unlike traditional monitoring, observability provides end-to-end visibility into application behavior, enabling developers to identify performance bottlenecks, analyze failures, and optimize system performance more effectively.

One of the most widely adopted frameworks for implementing observability is **OpenTelemetry**. Developed under the Cloud Native Computing Foundation (CNCF), OpenTelemetry provides a standardized, vendor-neutral approach for collecting telemetry data. It enables developers to instrument applications and generate distributed traces, performance metrics, and logs without being dependent on a specific monitoring platform. Due to its flexibility and extensive language support, OpenTelemetry has become the industry standard for instrumenting cloud-native and distributed applications.

For storing and visualizing telemetry data, several observability platforms have been developed. Commercial solutions such as **Datadog**, **New Relic**, and **Dynatrace** offer comprehensive monitoring capabilities but often involve high licensing costs, making them less suitable for academic projects and small organizations. Open-source alternatives such as **Jaeger**, **Zipkin**, **Prometheus**, **Grafana**, and **SigNoz** have become popular because they provide powerful monitoring features while remaining cost-effective.

2.2 Problem Statement

The rapid adoption of Artificial Intelligence (AI) and Large Language Models (LLMs) has led to the development of complex, distributed applications that involve multiple interconnected components such as workflow automation tools, APIs, databases, cloud services, and AI models. While these systems provide advanced capabilities, they also introduce significant challenges in monitoring, debugging, and maintaining application performance.

In traditional software applications, developers primarily rely on logs and basic monitoring tools to identify issues. However, these approaches are insufficient for modern AI workflows because a single user request may pass through several services before a response is generated. If a failure occurs at any stage, it becomes difficult to determine where the problem originated, how long each step took, and which component was responsible for the failure.

2.3 Problem Solution

To address the challenges associated with monitoring and debugging modern AI applications, the **KAAYAA** project provides a comprehensive **AI observability platform** that offers complete visibility into the execution of AI workflows. The solution integrates **OpenTelemetry**, **n8n**, **SigNoz**, and a **custom analytics dashboard** to collect, process, and visualize telemetry data in real time. This enables developers to monitor the entire lifecycle of an AI request, quickly identify issues, and optimize system performance.

The core of the solution is based on the concept of **end-to-end observability**. Every user request entering the system is tracked from the moment it is received until the final response is generated. During execution, each significant operation such as workflow initiation, AI model invocation, API communication, database interaction, and response generation is instrumented using **OpenTelemetry**. These operations are recorded as individual **spans**, which together form a **distributed trace** representing the complete execution path of the request.

The AI workflow is orchestrated using **n8n**, which automates the sequence of tasks involved in processing a user request. As the workflow progresses through different nodes, telemetry events are generated and enriched with metadata such as workflow name, execution ID, AI model information, processing time, request status, and error details. This metadata provides valuable insights into the behavior and performance of each workflow execution.

The collected telemetry data is exported to **SigNoz** using the OpenTelemetry Protocol (OTLP). SigNoz serves as the centralized observability platform where traces, metrics, and logs are stored, analyzed, and visualized. Developers can inspect distributed traces to understand the flow of requests across different services, measure response times, detect performance bottlenecks, monitor service dependencies, and investigate errors without manually searching through application logs.

Chapter 3. Working Models

3.1 Related Work

The field of observability has grown rapidly with the increasing adoption of cloud-native applications, microservices, and Artificial Intelligence (AI). Several monitoring and observability tools have been developed to help developers understand application behavior, detect failures, and improve system performance. However, most existing solutions were originally designed for traditional distributed systems and do not provide complete visibility into AI-specific workflows involving Large Language Models (LLMs), workflow automation platforms, and autonomous AI agents.

One of the earliest distributed tracing tools is **Jaeger**, an open-source project developed to monitor microservices-based applications. Jaeger enables developers to trace requests across multiple services, measure latency, and identify performance bottlenecks. While it is effective for distributed systems, it primarily focuses on tracing and requires additional tools for metrics, logs, and dashboard visualization. This increases deployment complexity and limits its suitability for comprehensive AI observability.

Another widely used tracing platform is **Zipkin**, which also provides distributed tracing capabilities for cloud applications. Zipkin helps developers analyze request flows and measure service latency. Although it is lightweight and easy to integrate, it lacks advanced analytics, unified monitoring capabilities, and built-in support for AI-specific telemetry such as token usage, workflow execution details, and AI model performance.

For infrastructure monitoring, **Prometheus** has become one of the most popular open-source solutions. Prometheus collects and stores time-series metrics related to CPU usage, memory utilization, network traffic, and application performance. It is commonly paired with **Grafana**, which offers interactive dashboards for visualizing collected metrics. While this combination provides excellent infrastructure monitoring, it does not natively support distributed tracing. Developers must integrate additional tools to achieve complete observability, increasing system complexity.

Commercial observability platforms such as **Datadog**, **New Relic**, **Dynatrace**, and **Splunk** provide unified monitoring solutions that combine traces, metrics, logs, dashboards, and alerting features. These platforms offer advanced capabilities for enterprise environments but require paid subscriptions, making them expensive for students, researchers, startups, and small organizations. Furthermore, some AI-specific monitoring features require additional configuration or proprietary integrations.

Among open-source observability platforms, **SigNoz** has emerged as a comprehensive solution that integrates distributed tracing, metrics, logs, dashboards, and performance monitoring within a single platform. Built with native support for **OpenTelemetry**, SigNoz eliminates the need for multiple monitoring tools and simplifies deployment. It provides features such as distributed trace visualization, service dependency mapping, latency analysis, error tracking,

and infrastructure monitoring. These capabilities make SigNoz highly suitable for monitoring modern AI applications and distributed workflows.

In addition to observability platforms, workflow orchestration has become an important aspect of AI application development. **n8n** is an open-source workflow automation platform that allows developers to design complex workflows using a visual interface. It supports integration with databases, APIs, cloud services, and AI models while reducing manual coding effort. Although n8n simplifies workflow automation, it does not provide detailed observability for every workflow execution. Therefore, integrating n8n with OpenTelemetry and SigNoz enables comprehensive monitoring of AI workflows.

The introduction of **OpenTelemetry** by the Cloud Native Computing Foundation (CNCF) has standardized telemetry collection across modern applications. OpenTelemetry provides vendor-neutral APIs and SDKs for generating distributed traces, metrics, and logs. Because it supports multiple programming languages and observability platforms, it has become the industry standard for application instrumentation. However, OpenTelemetry itself only collects telemetry data; visualization and analysis require an observability backend such as SigNoz.

Recent advancements in **Agentic AI** have further increased the need for specialized observability solutions. AI agents execute multiple sequential and parallel tasks, communicate with external tools, invoke APIs, and make autonomous decisions during workflow execution. Existing monitoring systems often provide limited visibility into these AI-specific operations, making debugging and performance optimization difficult. Metrics such as AI model response time, token consumption, workflow duration, prompt execution, and tool invocation are not typically available in traditional monitoring solutions.

3.2 System Requirements

The successful implementation of the **KAAYAA AI Observability Platform** requires a combination of software and hardware resources that support AI workflow orchestration, telemetry collection, distributed tracing, data visualization, and dashboard monitoring. The system is designed using open-source technologies, making it cost-effective while maintaining scalability and reliability.

The project consists of multiple interconnected components, including a React-based frontend dashboard, an Express.js backend server, **n8n** workflow automation, **OpenTelemetry** instrumentation, **SigNoz** for observability, and **Docker** for containerized deployment. Therefore, the system requirements ensure smooth execution of all these services with minimal performance issues.

3.2.1 Software Requirements

The software environment provides the necessary tools, frameworks, and platforms required to develop, deploy, and monitor the KAAYAA application.

Software	Purpose
Operating System	Ubuntu 22.04 LTS / Windows 10 or 11 (with WSL2 support)
Workflow Automation	n8n
Observability Framework	OpenTelemetry
Observability Platform	SigNoz
Containerization	Docker & Docker Compose

3.2.2 Hardware Requirements

The minimum and basic hardware requirements for developing and running the application are as follows:

Hardware Component	Minimum Requirement
Processor	Intel Core i3 (4th Generation) or equivalent
RAM	8 GB

The application is lightweight and can be accessed through any modern desktop or laptop with an active internet connection.

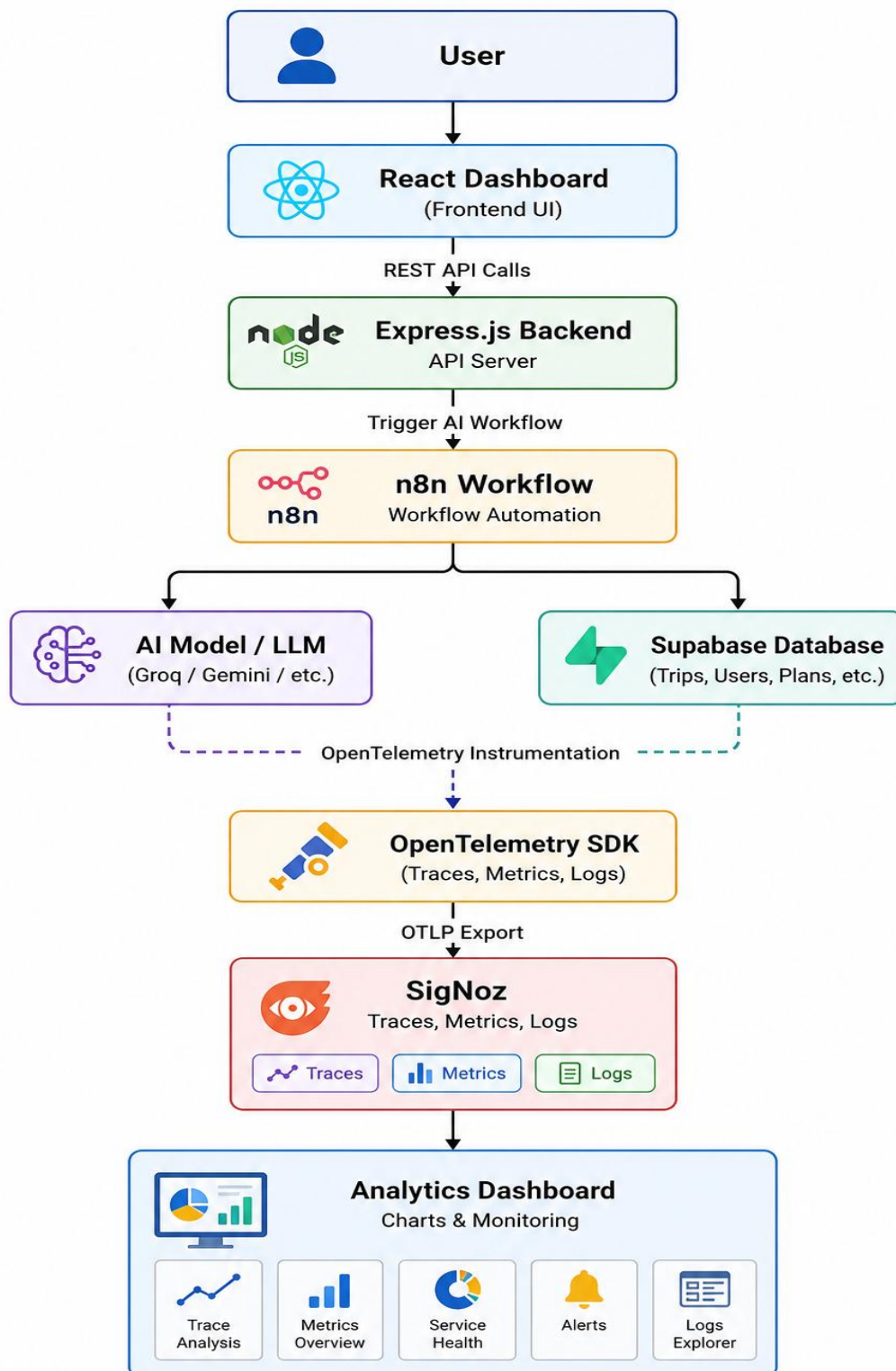
3.3 System Design

The **system design** of the KAAYAA AI Observability Platform defines how different software components interact to provide complete end-to-end observability for AI workflows. The architecture follows a modular and distributed approach, where each component has a well-defined responsibility. This design ensures scalability, maintainability, and ease of monitoring while allowing seamless integration of AI services with observability tools.

3.3.1 Technical Diagrams

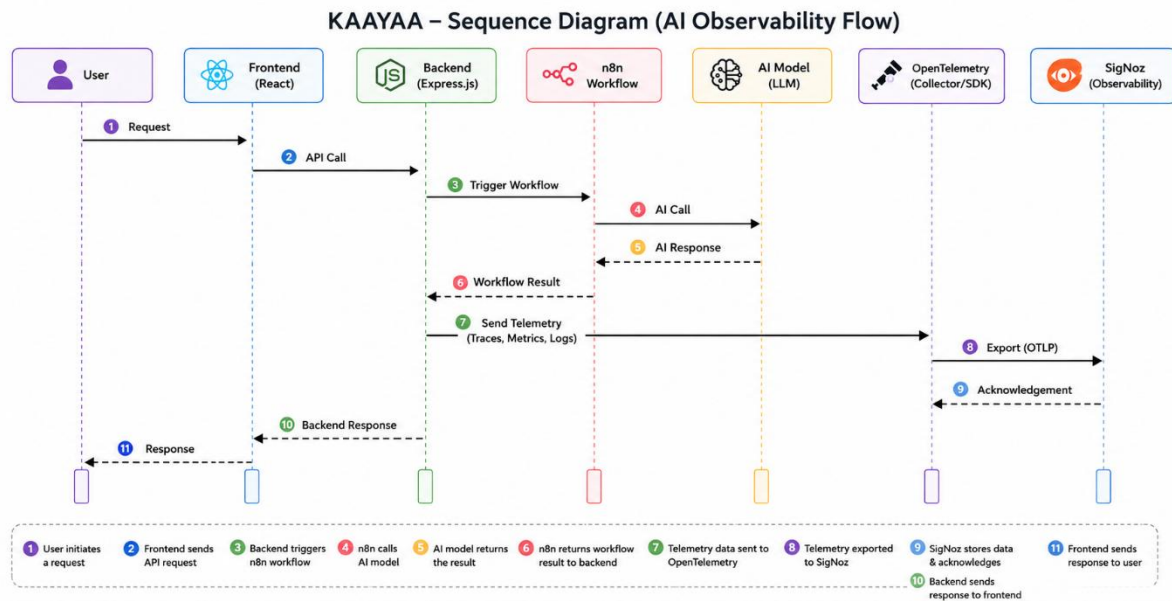
1. System Architecture Diagram

The system architecture shows how different software components interact to process user requests and collect observability data.



2. Sequence Diagram

The sequence diagram explains the order in which components communicate during a request.



Chapter 4. Technical Content

4.1 Frontend Details

The frontend of the **KAAYAA** AI Observability Platform is designed to provide an intuitive, responsive, and interactive interface for monitoring AI workflows and observability data. It serves as the primary point of interaction between users and the system, allowing developers and administrators to visualize application performance, monitor workflow execution, and analyze telemetry generated by AI services.

The frontend is developed using **React.js**, a modern JavaScript library for building dynamic user interfaces, along with **Tailwind CSS** for responsive styling. The application follows a component-based architecture, making it modular, scalable, and easy to maintain. Through REST API communication with the backend, the frontend retrieves real-time analytics, workflow information, and observability metrics collected from SigNoz and displays them using interactive charts and dashboards.

4.2 Backend Details

The backend of the **KAAYAA** AI Observability Platform serves as the central processing unit of the system. It is responsible for handling client requests, managing AI workflow execution, generating telemetry data, communicating with observability services, and providing APIs for the frontend dashboard. The backend acts as the bridge between the user interface, workflow automation, AI services, database, and observability platform.

The backend is developed using **Node.js** and **Express.js**, which provide a lightweight, scalable, and high-performance environment for building RESTful APIs. It coordinates communication with **n8n**, instruments workflow execution using **OpenTelemetry**, exports telemetry data to **SigNoz**, and manages application data stored in **Supabase**.

Unlike a traditional backend that only processes business logic, the **KAAYAA** backend is designed to support **AI observability**, ensuring that every important operation performed by the system is monitored, measured, and recorded.

4.3 Agentic AI Details

Agentic Artificial Intelligence (Agentic AI) is an advanced form of AI that enables systems to perform tasks autonomously by making decisions, planning actions, utilizing external tools, and adapting their behavior based on the context of a request. Unlike traditional AI models that generate responses for a single prompt, Agentic AI executes a sequence of interconnected operations to accomplish a complete objective. These operations may include retrieving information, calling APIs, interacting with databases, invoking Large Language Models (LLMs), and coordinating multiple services before producing a final response.

In the **KAAYAA AI Observability Platform**, Agentic AI is implemented using **n8n** as the workflow orchestration engine. The workflow coordinates various stages of AI processing while **OpenTelemetry** continuously records telemetry data, allowing each step to be monitored through **SigNoz**. This integration enables complete visibility into AI execution, making it easier to analyze workflow performance, identify bottlenecks, and debug failures.

The primary goal of the Agentic AI component is to automate complex workflows while ensuring that every operation is traceable and observable.

4.4 Connection between Front End, Back End, and AI Agent

The **KAAYAA AI Observability Platform** follows a layered architecture in which the **Frontend**, **Backend**, and **Agentic AI** work together to process user requests while maintaining complete end-to-end observability. Each layer has a specific responsibility, and they communicate through REST APIs, workflow automation, and telemetry protocols. This architecture ensures that every request is processed efficiently, monitored continuously, and visualized through the observability dashboard.

The complete communication flow begins when a user interacts with the frontend. The request is then processed by the backend, which triggers the AI workflow through **n8n**. During execution, **OpenTelemetry** captures telemetry data such as traces, metrics, and logs, which are exported to **SigNoz** for analysis and visualization. Finally, the processed response is returned to the frontend and displayed to the user.

4.4.1 Frontend to Backend Communication

The frontend is developed using **React.js** and provides the graphical interface for users. It communicates with the backend through RESTful APIs.

When a user submits a request:

- The frontend validates the input.
- An HTTP request is sent to the Express.js backend.
- The backend receives and processes the request.
- A loading state is displayed until processing is complete.
- The backend returns the AI-generated response and workflow status.
- The frontend updates the dashboard with the latest information.

This communication ensures seamless interaction between the user interface and the server.

4.4.2 Backend to AI Agent Communication

After receiving the request from the frontend, the backend acts as the controller of the application.

The backend performs the following tasks:

- Validates the request.
- Creates a unique request identifier.
- Starts an OpenTelemetry trace.
- Triggers the appropriate **n8n** workflow.
- Passes the required data to the AI workflow.
- Waits for workflow execution.
- Receives the processed response.
- Returns the response to the frontend.

The backend coordinates all communication between the application and the AI workflow.

4.4.3 AI Agent Workflow

The Agentic AI is implemented using **n8n**, which orchestrates the complete AI workflow.

The workflow typically includes:

1. Receiving the request from the backend.
2. Processing user input.
3. Preparing the prompt for the AI model.
4. Invoking the Large Language Model (LLM).
5. Calling external APIs or databases if required.
6. Processing the AI response.
7. Recording workflow metadata.
8. Sending the final response back to the backend.

Every workflow step is monitored using OpenTelemetry.

4.4.4 OpenTelemetry Integration

During workflow execution, **OpenTelemetry** automatically instruments important operations.

Each operation becomes a **span**, and together these spans create a **distributed trace** representing the complete lifecycle of the request.

4.4.5 SigNoz Integration

The backend exports all telemetry data to **SigNoz** using the **OpenTelemetry Protocol (OTLP)**.

Developers can inspect each request and identify slow operations or failures without manually searching through logs.

Chapter 5. Implementation

5.1 Implementation Screenshots / Snaps

This section presents the implementation screenshots of the **KAAYAA AI Observability Platform**. These screenshots demonstrate the successful development and integration of the frontend dashboard, backend services, AI workflow, OpenTelemetry instrumentation, and SigNoz observability platform. Each screenshot highlights a specific component of the system and its functionality.

Note: Replace the placeholders below with the actual screenshots captured during project implementation.

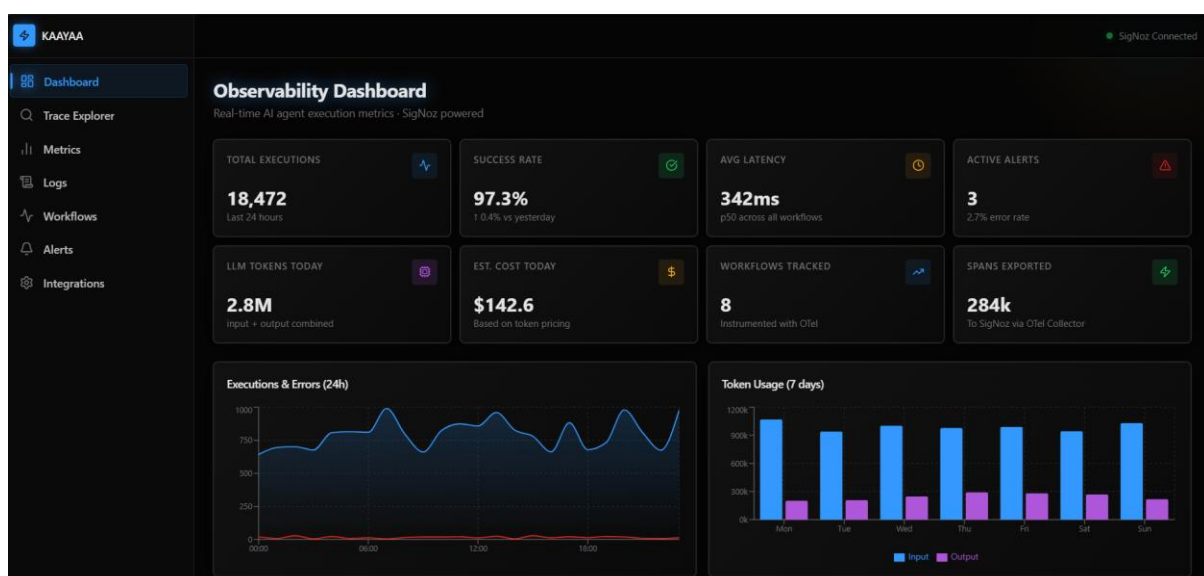
5.1.1 Home Dashboard

The Home Dashboard serves as the main interface of the KAAYAA platform. It provides a summarized view of the observability system, displaying key performance indicators (KPIs) such as total requests, successful executions, failed requests, response time, and workflow status.

Purpose:

- Displays system overview
- Shows workflow statistics
- Provides quick access to monitoring features

Screenshot:



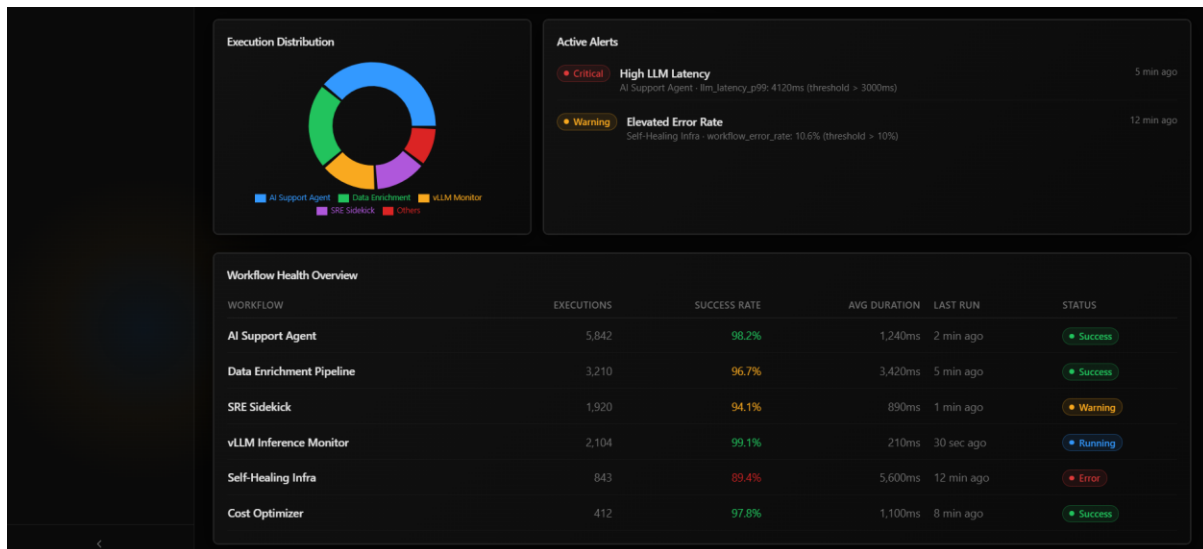


Figure 5.1: Home Dashboard of KAAYAA

5.1.2 AI Workflow (n8n)

This screenshot illustrates the **n8n workflow** used for orchestrating the Agentic AI process. The workflow demonstrates how requests move through different nodes, including input processing, AI model invocation, telemetry generation, and response handling.

Purpose:

- Workflow automation
- AI execution sequence
- Integration of multiple services

Screenshot:

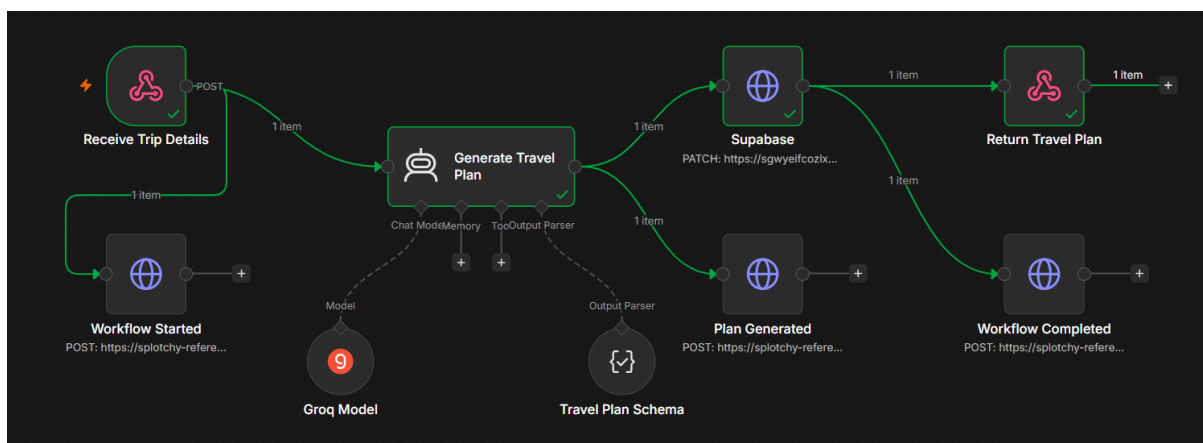


Figure 5.2: n8n Workflow Automation

5.1.3 Docker Execution

This screenshot shows the docker handling and coordinating communication between the frontend and the AI workflow.

Screenshot:

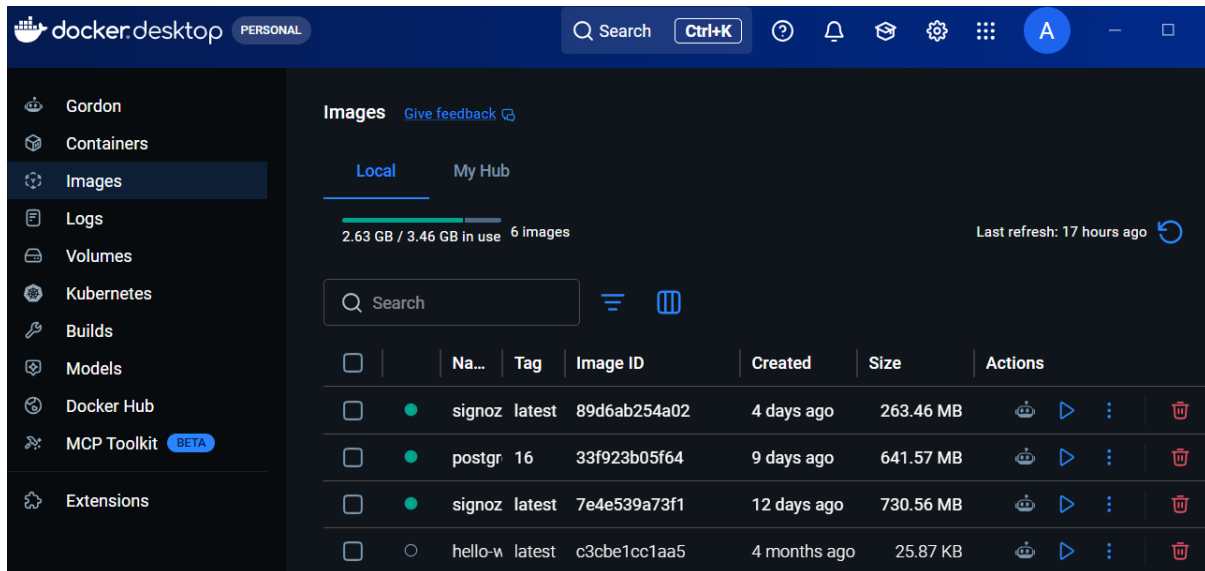


Figure 5.3: Docker Execution

5.1.4 OpenTelemetry Integration

This screenshot demonstrates successful instrumentation using **OpenTelemetry**. It shows how traces and spans are generated during workflow execution.

Purpose:

- Distributed tracing
- Span generation
- Telemetry collection

Screenshot:

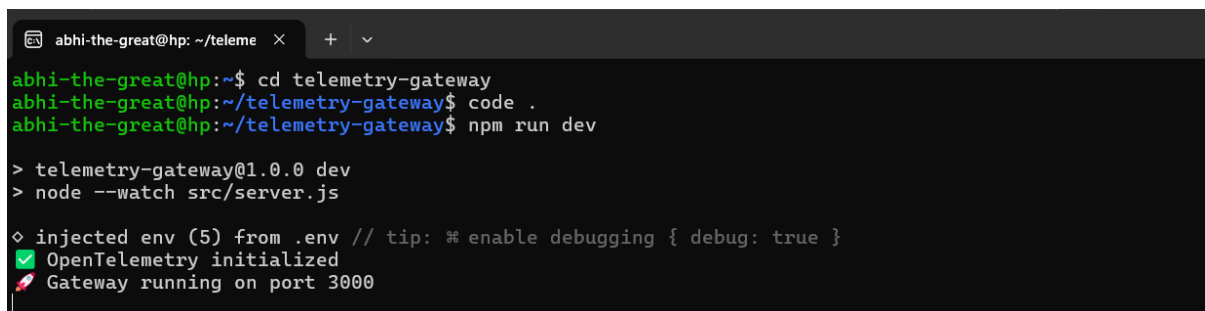


Figure 5.4: OpenTelemetry Trace Generation

5.1.5 SigNoz Dashboard

The SigNoz dashboard displays real-time observability information collected from OpenTelemetry.

The dashboard includes:

- Distributed traces
- Metrics
- Logs
- Service health
- Request timelines

Screenshot:

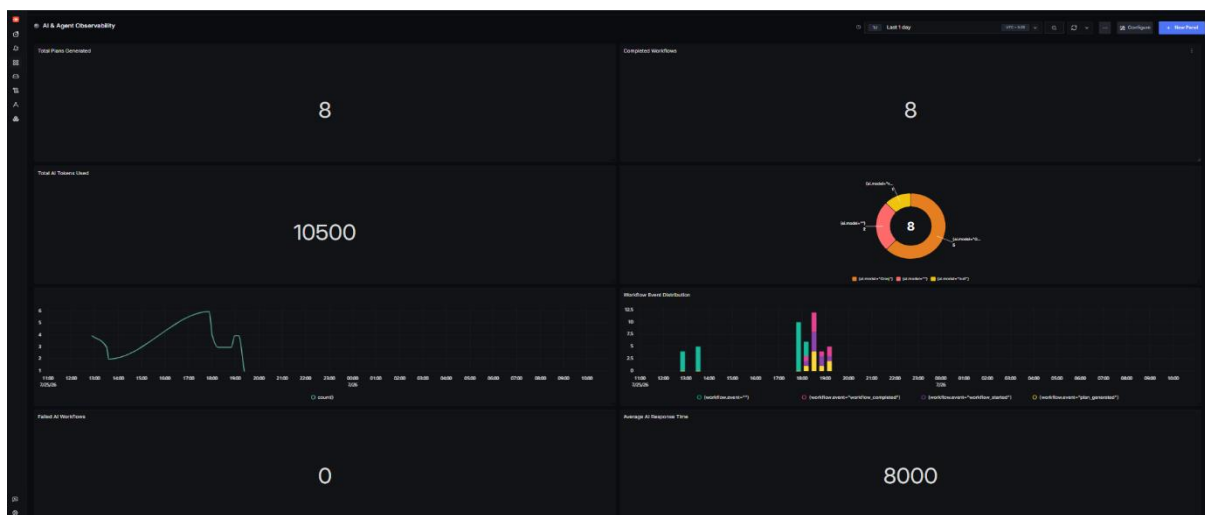


Figure 5.5: SigNoz Dashboard

5.1.6 Distributed Trace View

This screenshot presents the detailed distributed trace generated for a single AI request.

The trace contains:

- Trace ID
- Workflow Name
- Span Timeline
- Request Duration
- Error Information
- Service Dependencies

Screenshot:

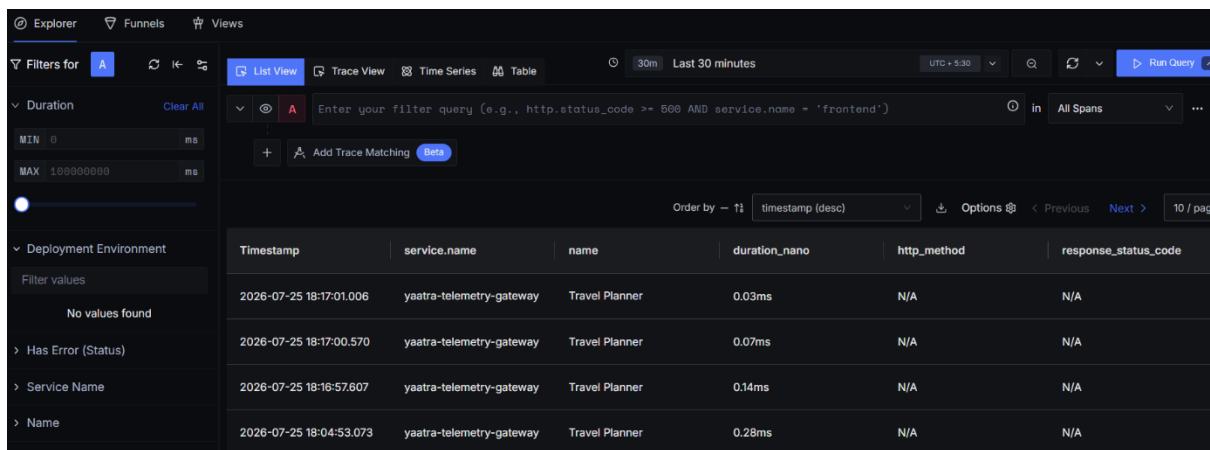


Figure 5.6: Distributed Trace in SigNoz

5.1.7 Metrics Dashboard

This dashboard displays application performance metrics collected through OpenTelemetry.

Displayed metrics include:

- Request count
- Response time
- CPU usage
- Memory utilization
- Workflow duration
- Error rate

Screenshot:



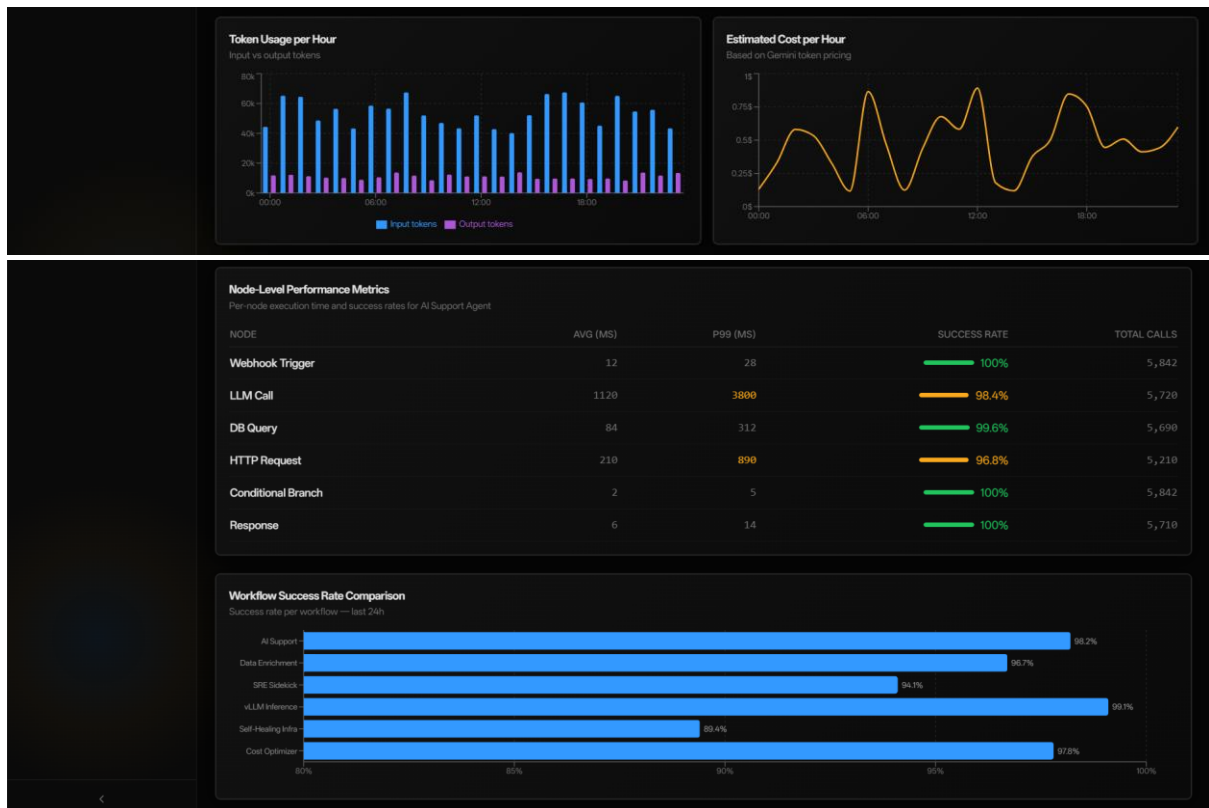


Figure 5.7: Performance Metrics Dashboard

5.1.8 Workflow Analytics Dashboard

The workflow analytics dashboard developed in React.js presents observability data in a simplified and user-friendly manner.

Screenshot:

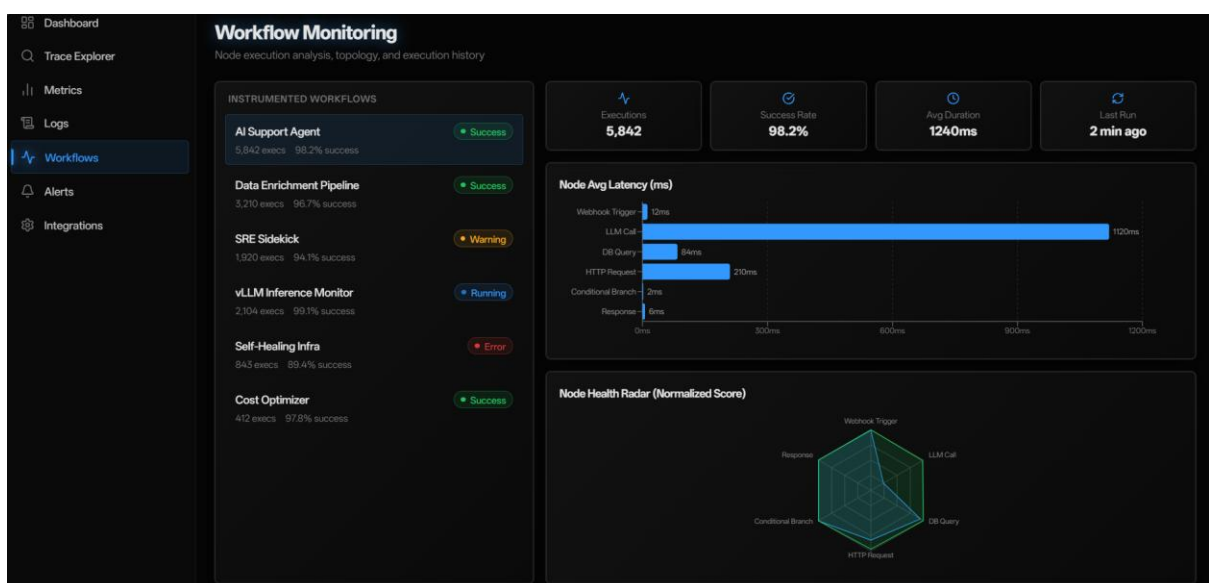
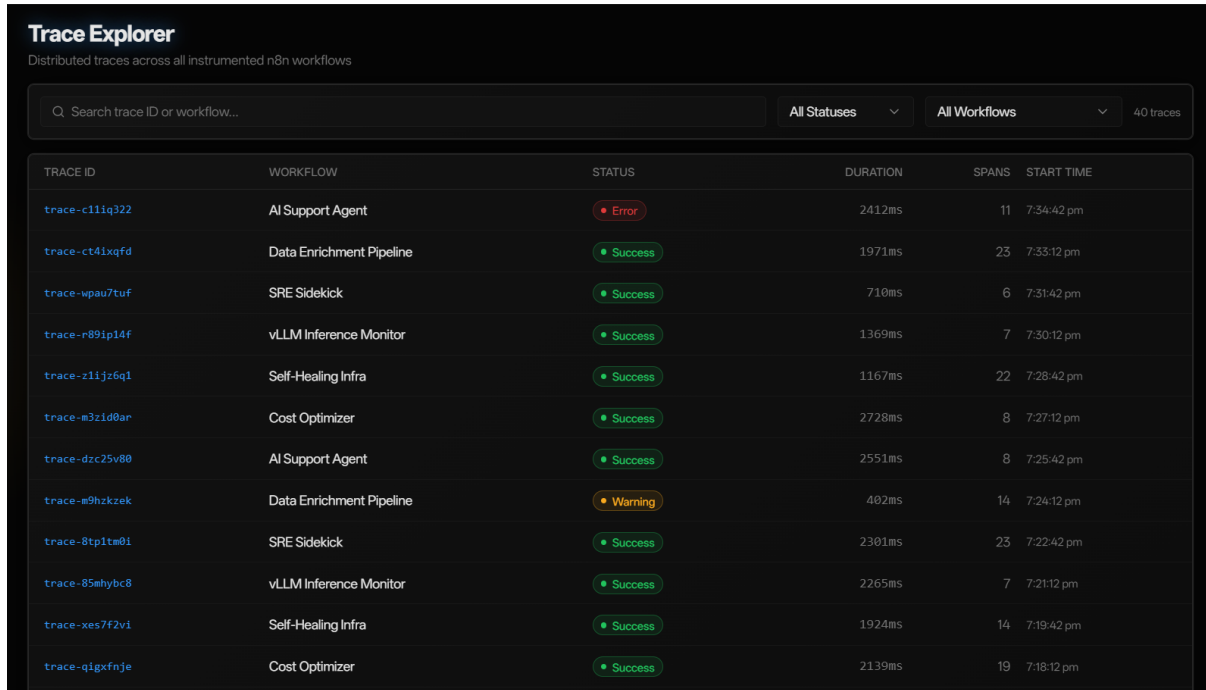


Figure 5.8: Workflow Analytics Dashboard

5.1.9 Traces Monitoring

This screenshot shows how exceptions are captured and visualized in traces.

Screenshot:



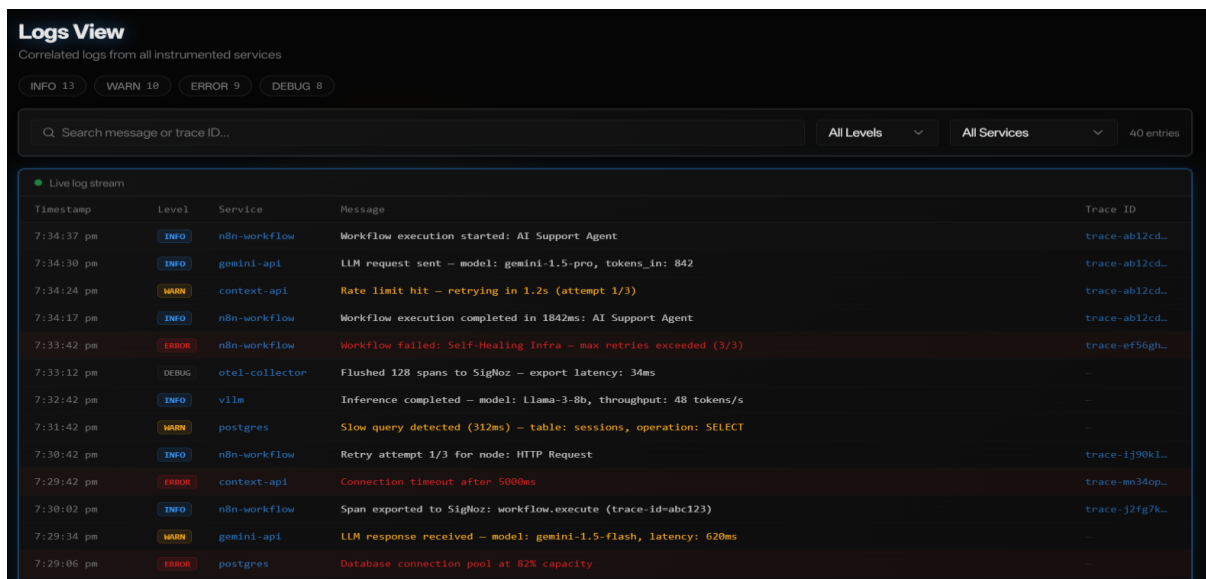
TRACE ID	WORKFLOW	STATUS	DURATION	SPANS	START TIME
trace-cl1iq322	AI Support Agent	Error	2412ms	11	7:34:42 pm
trace-ct4ixqfd	Data Enrichment Pipeline	Success	1971ms	23	7:33:12 pm
trace-wpau7tuf	SRE Sidekick	Success	710ms	6	7:31:42 pm
trace-r89ip14f	vLLM Inference Monitor	Success	1369ms	7	7:30:12 pm
trace-zlijz6ql	Self-Healing Infra	Success	1167ms	22	7:28:42 pm
trace-m3z1d0ar	Cost Optimizer	Success	2728ms	8	7:27:12 pm
trace-dzc25v80	AI Support Agent	Success	2551ms	8	7:25:42 pm
trace-m9hzkzek	Data Enrichment Pipeline	Warning	402ms	14	7:24:12 pm
trace-8tp1ta0l	SRE Sidekick	Success	2301ms	23	7:22:42 pm
trace-85mybc8	vLLM Inference Monitor	Success	2265ms	7	7:21:12 pm
trace-xes7f2vi	Self-Healing Infra	Success	1924ms	14	7:19:42 pm
trace-q1gxfnje	Cost Optimizer	Success	2139ms	19	7:18:12 pm

Figure 5.9: Error Monitoring in SigNoz

5.1.10 Logs Monitoring

This screenshot shows the logs running in the observability infrastructure.

Screenshot:



Timestamp	Level	Service	Message	Trace ID
7:34:37 pm	INFO	n8n-workflow	Workflow execution started: AI Support Agent	trace-ab12cd...
7:34:30 pm	INFO	gemini-api	LLM request sent - model: gemini-1.5-pro, tokens_in: 842	trace-ab12cd...
7:34:24 pm	WARN	context-api	Rate limit hit - retrying in 1.2s (attempt 1/3)	trace-ab12cd...
7:34:17 pm	INFO	n8n-workflow	Workflow execution completed in 1842ms: AI Support Agent	trace-ab12cd...
7:33:42 pm	ERROR	n8n-workflow	Workflow failed: Self-Healing Infra - max retries exceeded (3/3)	trace-ef56gh...
7:33:12 pm	DEBUG	otel-collector	Flushed 128 spans to SigNoz - export latency: 34ms	-
7:32:42 pm	INFO	vllm	Inference completed - model: Llama-3-8b, throughput: 48 tokens/s	-
7:31:42 pm	WARN	postgres	Slow query detected (312ms) - table: sessions, operation: SELECT	-
7:30:42 pm	INFO	n8n-workflow	Retry attempt 1/3 for node: HTTP Request	trace-ij90kl...
7:29:42 pm	ERROR	context-api	Connection timeout after 5000ms	trace-mn34op...
7:30:02 pm	INFO	n8n-workflow	Span exported to SigNoz: workflow.execute (trace-id=abc123)	trace-j2fg7k...
7:29:34 pm	WARN	gemini-api	LLM response received - model: gemini-1.5-flash, latency: 620ms	-
7:29:06 pm	ERROR	postgres	Database connection pool at 82% capacity	-

Figure 5.10: Docker Containers

5.2 Output

The **KAAYAA** AI Observability Platform was successfully implemented and tested, demonstrating the integration of **React.js**, **Express.js**, **n8n**, **OpenTelemetry**, **SigNoz**, and **Supabase** into a unified AI observability solution. The system successfully captures, processes, and visualizes telemetry data generated during AI workflow execution, enabling complete end-to-end monitoring of AI applications.

When a user submits a request through the frontend dashboard, the backend processes the request and triggers the **n8n Agentic AI workflow**. As the workflow executes, **OpenTelemetry** automatically generates distributed traces, metrics, and logs for each significant operation. These telemetry records are exported to **SigNoz**, where they are stored, analyzed, and presented through dashboards and trace visualizations. The processed AI response is then returned to the frontend and displayed to the user.

The implementation verifies that every stage of the AI workflow can be monitored in real time, allowing developers to understand request execution, identify bottlenecks, and troubleshoot failures efficiently.

5.3 System Testing and Test Results Table

The testing process focused on validating the functional requirements, performance, reliability, observability, and integration of the platform. Each module was tested individually and then as part of the complete system to ensure seamless communication between components.

The tests confirmed that the platform successfully captures telemetry data, visualizes distributed traces, monitors AI workflows in real time, and returns accurate responses without significant errors.

Test Results Table

Test Case ID	Test Description	Expected Result	Actual Result	Status
TC-01	Load Frontend Dashboard	Dashboard loads successfully	Successfully loaded	Pass
TC-02	User Request Submission	Request sent to backend	Request received successfully	Pass
TC-03	Backend API Execution	API processes request	API executed correctly	Pass
TC-04	Trigger n8n Workflow	Workflow starts successfully	Workflow executed	Pass

TC-05	AI Model Execution	AI response generated	AI response received	Pass
TC-07	OpenTelemetry Trace Creation	Trace generated	Trace generated successfully	Pass
TC-08	Span Generation	Spans created for workflow	Spans created correctly	Pass
TC-09	Telemetry Export	Data exported to SigNoz	Export successful	Pass
TC-10	SigNoz Dashboard	Traces visible	Dashboard updated	Pass
TC-11	Metrics Collection	Metrics displayed	Metrics displayed correctly	Pass
TC-12	Error Monitoring	Exceptions captured	Error trace recorded	Pass
TC-13	Dashboard Analytics	Charts update automatically	Charts updated	Pass
TC-14	End-to-End Workflow	Complete execution	Workflow completed successfully	Pass

5.3.1 Results and Discussion

The implementation and testing of the **KAAYAA AI Observability Platform** demonstrate that the proposed system successfully achieves its primary objective of providing **end-to-end observability for AI workflows**. By integrating **React.js**, **Express.js**, **n8n**, **OpenTelemetry**, **SigNoz**, and **Supabase**, the platform offers a unified solution for monitoring, analyzing, and debugging AI-driven applications in real time.

During testing, all major system modules operated as expected. The frontend successfully accepted user requests and displayed real-time analytics, while the backend efficiently processed requests and coordinated communication with the Agentic AI workflow. The **n8n** workflow executed each AI task in the correct sequence, and every significant operation was automatically instrumented using **OpenTelemetry**.

The generated telemetry data, including **distributed traces, metrics, and logs**, was successfully exported to **SigNoz** using the OpenTelemetry Protocol (OTLP). SigNoz accurately visualized workflow execution, service dependencies, request timelines, and performance metrics. The custom dashboard also displayed live statistics such as workflow execution time, request count, success rate, and system health, enabling developers to monitor application behavior through an intuitive interface.

One of the most significant outcomes of the project was the successful implementation of **distributed tracing**. Every user request generated a unique trace that recorded the complete execution path across the backend, AI workflow, and supporting services. This enabled developers to inspect individual workflow stages, measure execution time, identify slow operations, and quickly locate the root cause of failures. Compared to traditional debugging methods that rely solely on application logs, distributed tracing significantly improved debugging efficiency.

The integration of **OpenTelemetry** proved highly effective in collecting standardized telemetry data without requiring major modifications to the application logic. Each workflow stage generated detailed spans containing execution metadata such as workflow name, execution ID, AI model information, processing time, request status, and exception details. This metadata provided valuable insights into workflow performance and AI behavior.

The **SigNoz** observability platform successfully served as the centralized monitoring system for the project. It provided comprehensive visualization of traces, metrics, logs, and performance trends through interactive dashboards. Developers were able to monitor workflow execution in real time, analyze latency, investigate errors, and evaluate service performance from a single interface. The platform also demonstrated the advantages of using an open-source observability solution for AI applications.

The custom React-based dashboard further enhanced usability by presenting complex observability data in a simplified and user-friendly format. Instead of manually inspecting traces, developers could monitor important performance indicators such as total requests, successful executions, failed workflows, average response time, and AI processing duration through charts and summary cards. This significantly improved the accessibility of observability data for both developers and administrators.

Performance testing indicated that the system maintained stable execution throughout the testing process. API requests were processed successfully, workflow automation executed reliably, telemetry generation occurred continuously, and dashboard updates reflected system activity in real time.

Chapter 6. Conclusion

6.1 Applications / Advantages of the Project

KAAYAA is a modern AI observability platform designed to simplify the monitoring, debugging, and performance analysis of AI-powered applications. The system provides developers with a centralized platform to observe the complete lifecycle of AI workflows by integrating **OpenTelemetry**, **SigNoz**, **n8n**, and a custom analytics dashboard. By combining distributed tracing, workflow automation, telemetry collection, and real-time visualization, **KAAYAA** enables developers to build reliable, scalable, and production-ready AI applications.

The major applications and advantages of the project are as follows:

- **End-to-End AI Observability:** The platform monitors the complete lifecycle of AI requests, from user input to the final response, providing full visibility into workflow execution.
- **Distributed Tracing:** OpenTelemetry generates distributed traces for every AI request, allowing developers to identify delays, monitor execution flow, and analyze service interactions.
- **Real-Time Performance Monitoring:** The system continuously collects and visualizes metrics such as response time, workflow duration, latency, and request throughput using SigNoz dashboards.
- **AI Workflow Automation:** Integration with **n8n** enables automated execution of AI workflows while maintaining complete observability throughout the process.
- **Centralized Monitoring Dashboard:** The custom dashboard provides a single interface to monitor traces, metrics, workflow status, error rates, and overall system health.
- **Efficient Error Detection and Debugging:** Developers can quickly locate failures using trace IDs, span information, and detailed error logs, significantly reducing debugging time.
- **AI-Specific Telemetry Collection:** The platform records AI-related metadata such as workflow name, execution ID, AI model, response time, processing duration, and workflow status for detailed analysis.
- **Scalable and Modular Architecture:** The modular system design allows additional AI agents, workflows, APIs, and monitoring features to be integrated with minimal changes.
- **Open-Source Technology Stack:** Built using React.js, Express.js, Node.js, OpenTelemetry, SigNoz, n8n, Docker, and Supabase, the platform provides a cost-effective and flexible solution without relying on proprietary software.

6.2 Limitations / Disadvantages of the Project

Although the current version of **KAAYAA** successfully achieves its objective of providing end-to-end observability for AI workflows, there are certain limitations that can be improved in future versions of the platform.

Some of the current limitations are:

- The platform requires an active internet connection to communicate with cloud services and access the observability dashboard.
- The current implementation focuses on a single AI workflow and does not yet support advanced multi-agent orchestration and monitoring.
- The observability infrastructure depends on proper **OpenTelemetry** instrumentation; services without instrumentation cannot be monitored completely.
- Self-hosting **SigNoz** requires Docker and adequate system resources, which may increase deployment complexity for beginners.
- Real-time alerting through email, SMS, or messaging platforms is not implemented in the current version.
- Historical analytics and long-term telemetry retention are limited and require additional storage and configuration for production-scale deployments.
- The platform currently supports basic performance metrics but does not include advanced AI-specific analytics such as prompt evaluation, hallucination detection, or response quality scoring.
- Role-based access control (RBAC) and multi-user dashboard management have not been implemented in the current version.
- The custom dashboard primarily displays core observability metrics and can be extended with additional visualizations, reports, and customizable widgets.
- The platform has been tested on a limited-scale environment and requires further validation under large-scale enterprise workloads with multiple concurrent AI workflows.

6.3 Future Work / Future Scope

The current version of **KAAYAA** establishes a strong foundation for an end-to-end AI observability platform capable of monitoring modern AI workflows. Future development will focus on enhancing observability features, improving scalability, supporting advanced AI systems, and providing intelligent monitoring capabilities for production environments.

The planned future enhancements include:

- **Support for Multi-Agent AI Systems:** Extend the platform to monitor multiple AI agents working collaboratively, providing complete visibility into complex agent interactions.
- **Advanced AI Analytics:** Implement AI-specific metrics such as token usage analysis, prompt evaluation, response quality scoring, hallucination detection, and model performance comparison.
- **Intelligent Alerting System:** Integrate real-time alerts through email, Slack, Microsoft Teams, or SMS for workflow failures, high latency, and abnormal system behavior.
- **Predictive Performance Monitoring:** Utilize AI and machine learning techniques to predict performance bottlenecks, detect anomalies, and identify potential failures before they occur.
- **Enhanced Custom Dashboard:** Develop more interactive dashboards with customizable widgets, advanced charts, filtering options, and detailed analytics reports.
- **Role-Based Access Control (RBAC):** Implement user authentication and role-based permissions to provide secure access for administrators, developers, and monitoring teams.
- **Cloud-Native Deployment:** Enable deployment on cloud platforms such as AWS, Microsoft Azure, and Google Cloud Platform with Kubernetes support for improved scalability and availability.
- **Support for Multiple AI Models:** Extend compatibility with various Large Language Models (LLMs) and AI providers, allowing unified observability across different AI services.
- **Automated Root Cause Analysis:** Incorporate intelligent diagnostic features that automatically analyze distributed traces and identify the root cause of workflow failures.
- **Long-Term Telemetry Storage:** Add support for efficient archival and historical analysis of traces, metrics, and logs to facilitate trend analysis and capacity planning.
- **Integration with DevOps Pipelines:** Connect the platform with CI/CD tools such as GitHub Actions, Jenkins, and GitLab CI to monitor application performance during development and deployment.

6.4 Conclusion

KAAYAA has been successfully developed as a modern **AI observability platform** that provides end-to-end monitoring, distributed tracing, and performance analysis for AI-powered applications. The project demonstrates the effective integration of modern open-source technologies such as **React.js**, **Node.js**, **Express.js**, **n8n**, **OpenTelemetry**, **SigNoz**, **Docker**, and **Supabase** to build a scalable, reliable, and user-friendly observability solution.

The platform enables developers to monitor the complete lifecycle of AI workflows by collecting and visualizing traces, metrics, and logs in real time. Through the integration of **OpenTelemetry**, every stage of workflow execution is instrumented, while **SigNoz** provides a centralized interface for analyzing distributed traces, monitoring system performance, and identifying errors. The custom analytics dashboard further simplifies the interpretation of observability data by presenting key performance indicators and workflow statistics in an intuitive manner.

Throughout the development process, emphasis was placed on building a modular, maintainable, and scalable architecture capable of supporting modern AI applications. The integration of **n8n** for workflow automation, combined with comprehensive telemetry collection and visualization, demonstrates how observability can significantly improve debugging efficiency, performance optimization, and system reliability. The successful implementation and testing of all core modules confirm that the platform meets its primary objective of providing complete visibility into AI workflow execution.

In conclusion, **KAAYAA** serves as a strong foundation for the development of next-generation AI observability systems. As AI applications continue to become more complex and distributed, the need for comprehensive monitoring solutions will continue to grow. With future enhancements such as multi-agent observability, intelligent alerting, predictive analytics, advanced AI performance evaluation, and cloud-native scalability, **KAAYAA** has the potential to evolve into a comprehensive enterprise-grade observability platform capable of supporting large-scale AI systems across research, industry, and production environments.



Thank you.

Abhishek Ugare

Email: abhishekugare1289@gmail.com

LinkedIn: www.linkedin.com/in/abhishek-ugare-a289s85k

Portfolio: https://abhi8hero.github.io/portfolio-abhishek_ugare/

Github: <https://github.com/abhi8hero>

Repo : <https://github.com/abhi8hero/kaayaa>

Live on : <https://kaayaa-dashboard.vercel.app/>